

INDIAN SIGN LANGUAGE (ISL) RECOGNITION SYSTEM

README / PROJECT DOCUMENTATION

Project type: Computer Vision + Deep Learning + Real-Time Human-Computer Interaction

Recognition approach: MediaPipe Holistic landmark extraction + Transformer-based sequence classification

Current prototype classes: A, B, C, D, E

Purpose

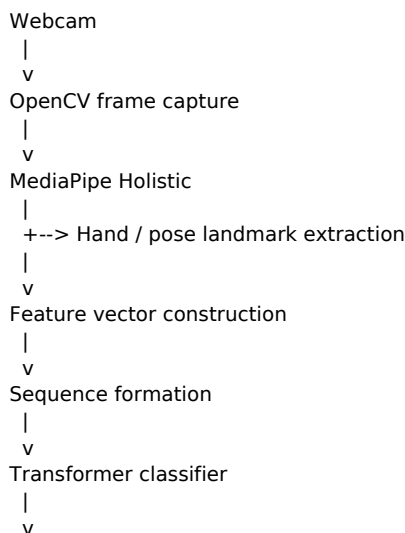
This project is a real-time Indian Sign Language recognition prototype designed to recognize hand signs from a webcam and convert the predicted alphabet into readable text and spoken output. The system uses landmark-based computer vision instead of relying directly on raw RGB images. MediaPipe Holistic extracts hand and body-related landmarks, a sequence model learns temporal patterns, and a Transformer classifier predicts the sign.

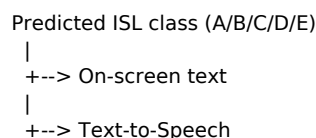
The project was developed and tested as a laptop/webcam-based system. The current verified prototype focuses on the ISL alphabet classes A-E and is structured so that additional signs can be added after collecting and training on sufficient data.

1. PROJECT OBJECTIVES

- Recognize Indian Sign Language alphabet gestures using a normal webcam.
- Use landmark-based features to reduce dependence on raw image appearance.
- Support dynamic/sequence-based recognition using a Transformer model.
- Support two-hand information through MediaPipe Holistic rather than limiting the pipeline to a single hand.
- Display the predicted alphabet in real time.
- Provide text-to-speech output for the recognized alphabet.
- Create a foundation that can later be expanded from A-E to a larger ISL vocabulary.

2. SYSTEM WORKFLOW





In the real-time stage, consecutive webcam frames are converted into landmark feature vectors. A fixed-length sequence is passed to the trained Transformer. The predicted class is then displayed and can be sent to the speech engine.

3. TECHNOLOGIES USED

Technology	Purpose
Python 3.11	Main programming language and runtime
OpenCV	Webcam capture, image processing and display
MediaPipe Holistic	Landmark extraction from the human body and hands
PyTorch	Transformer model training and inference
NumPy	Numerical processing and feature handling
scikit-learn	Supporting preprocessing/evaluation utilities
pyttsx3	Offline text-to-speech output

4. DATASET USED

The prototype dataset was created from the project's own recorded ISL alphabet videos. The videos were organized by alphabet class and processed into sequences of landmark features.

Item	Recorded / observed project result
Sign classes	A, B, C, D, E
Source	User-recorded ISL alphabet videos
Example naming	A.mp4, A1.mp4, A2.mp4 and corresponding files for other classes
Initial sequence extraction run	23 videos produced 82 usable sequences
Sequence length	30 frames per sequence
Feature size in the earlier Transformer	223 features
Class counts in that run	A: 34, B: 11, C: 11, D: 7, E: 19
Validation approach	4-fold video-level cross-validation in the earlier run
Later feature pipeline	MediaPipe Holistic-based 260-feature representation

Dataset note

The dataset is relatively small and class-balanced only approximately. In particular, class D had fewer extracted sequences than the other classes. Therefore, the current model should be treated as a working prototype rather than a production-grade ISL recognizer. More videos, different users, different hand orientations, backgrounds and lighting conditions are required for robust generalization.

5. FEATURE EXTRACTION

The project evolved from a simpler hand-landmark pipeline to a MediaPipe Holistic pipeline. The earlier run used a 223-dimensional feature vector. The later target pipeline uses a 260-dimensional

representation so that more holistic landmark information can be retained.

- Webcam frames are captured using OpenCV.
- MediaPipe processes each frame and extracts landmark coordinates.
- Relevant landmark coordinates are flattened into a feature vector.
- Feature normalization is applied using the saved training mean and standard deviation.
- A sequence of normalized feature vectors is provided to the Transformer.

6. TRANSFORMER MODEL

The recognition model is sequence-based. Instead of treating every webcam frame as an independent classification problem, the model receives a short temporal sequence. This allows it to learn how landmark positions change across frames while a sign is being performed.

- Input sequence length used in the prototype: 30 frames.
- The earlier model used 223 features per frame.
- The later Holistic pipeline uses a 260-feature input representation.
- Training normalization parameters are saved and reused during inference.
- The real-time inference code loads the trained model and normalization data before opening the webcam.
- The prototype runs on CPU and was tested with PyTorch CPU inference.

7. REAL-TIME IMPLEMENTATION

The real-time application was successfully brought to the stage where the Transformer loads, normalization data loads, MediaPipe initializes, and the webcam provides frames. The project logs confirmed the webcam opened and frames were received.

```
Transformer loaded successfully.  
Normalization loaded.
```

```
Starting TTS...  
Starting MediaPipe...
```

```
Opening webcam...  
Camera opened: True  
Frame received: True
```

The runtime also produced normal MediaPipe/TensorFlow Lite informational and warning messages, including XNNPACK initialization and feedback-manager warnings. These messages do not by themselves indicate that the recognition pipeline has failed.

8. TEXT-TO-SPEECH

Text-to-speech was added using pyttsx3 so that the predicted alphabet can be spoken aloud. The speech component is intended to speak the current recognized class rather than repeatedly spelling the same letter.

Important current limitation: During testing, the speech output was observed to repeatedly speak the letter E even when other letters were expected. This indicates that the TTS engine itself can produce speech, but the prediction-to-speech logic and/or model confidence/stability still needs refinement for reliable A/B/C/D/E switching. The issue should therefore be treated as a recognition/debouncing/inference problem rather than simply a missing TTS library.

- Recommended improvement: require the same prediction for several consecutive frames/sequences before speaking.

- Add a confidence threshold before accepting a prediction.
- Add a cooldown after speech to prevent repeated announcements.
- Log predicted class and confidence to determine whether the model itself is producing E continuously.

9. PROJECT FILE STRUCTURE

```

Signlanguagerobot/
|
├── 1_extract_holistic.py
├── 2_train_holistic_transformer.py
├── 2_train_model.py
├── 3_realtime_transformer.py
├── 4_realtime_transformer.py
├── betteraccuracy.py
├── extract_transformer_data.py
├──
└── Videos/
    ├── A.mp4
    ├── A1.mp4
    ├── A2.mp4
    ├── B.mp4
    ├── ...
    └── E*.mp4

```

The exact generated model/checkpoint and normalization filenames can vary between training versions. The real-time script should therefore be configured to point to the checkpoint and normalization files actually produced by the selected training run.

10. MAIN PROJECT SCRIPTS

File	Role
1_extract_holistic.py	Extracts Holistic landmark features from the recorded videos.
2_train_holistic_transformer.py	Training pipeline for the Holistic Transformer model.
2_train_model.py	Earlier/model-training pipeline used during development.
3_realtime_transformer.py	Real-time Transformer inference version.
4_realtime_transformer.py	Later real-time Transformer implementation used during testing.
extract_transformer_data.py	Utility for creating Transformer-ready sequence data.
betteraccuracy.py	Experimental accuracy/improvement work used during development.

11. INSTALLATION

A Python 3.11 environment was used for the project. Install the required packages with:

```
pip install opencv-python mediapipe numpy torch scikit-learn pytsx3
```

If a package is already installed, pip may report it as already satisfied. The exact PyTorch build can be CPU-only when a GPU is not available.

12. HOW TO RUN

Open PowerShell and move into the project directory:

```
cd "C:\Users\Asmita\OneDrive\Desktop\Signlanguagerobot"
```

For training, first extract the landmark sequences and then run the selected Transformer training script. For real-time testing, run the real-time script corresponding to the trained checkpoint, for example:

```
python 4_realtime_transformer.py
```

The exact command should match the filename present in the project folder. Before running, confirm that the model checkpoint and normalization files referenced by the script exist.

13. EXPECTED REAL-TIME OPERATION

- The program starts and loads the Transformer checkpoint.
- Normalization parameters are loaded.
- MediaPipe Holistic starts.
- The webcam opens.
- The application collects enough frames to form a sequence.
- The Transformer predicts one of the trained classes A-E.
- The predicted alphabet is displayed.
- The speech engine announces the accepted prediction, subject to the confidence/cooldown logic.

14. DEVELOPMENT RESULTS

The project successfully progressed from a basic landmark-recognition concept to a sequence-based Transformer pipeline with a Holistic landmark representation and real-time webcam inference.

Area	Result / status
Webcam capture	Working; camera opened and frames were received during testing.
MediaPipe initialization	Working in the tested real-time pipeline.
Transformer loading	Working; checkpoint loaded successfully.
Normalization loading	Working.
Real-time display	Working in the tested implementation.
Text-to-speech engine	Integrated and able to produce speech.
A-E classification	Prototype classes implemented.
Speech class stability	Needs improvement; E was repeatedly spoken during one testing stage.
Two-hand/holistic representation	Supported by the later 260-feature pipeline.

15. KNOWN LIMITATIONS

- The current dataset contains only A-E, so this is not yet a complete ISL alphabet recognizer.
- The number of recorded samples is small for a deep-learning model.
- The classes are not equally represented in the extracted sequence data.
- Performance can change with camera angle, lighting, background, distance and hand orientation.
- A prediction model can confuse visually similar gestures when training data is limited.
- Repeated E speech observed during testing shows that prediction stability and speech gating need improvement.
- The system should not be presented as a complete or medically/officially certified communication system.

16. RECOMMENDED IMPROVEMENTS

- Record substantially more samples for every class.
- Use multiple people as signers to improve generalization.

- Capture different lighting, backgrounds, distances and camera angles.
- Balance the number of sequences across classes.
- Use confidence thresholds and consecutive-frame voting.
- Use a speech cooldown so one recognized sign is not spoken repeatedly.
- Log prediction probabilities during testing.
- Evaluate with a signer-independent test set instead of testing only on recordings similar to training data.
- Expand from A-E to the complete ISL alphabet and then to common words/phrases.
- Consider a word-level sequence model after the alphabet-level system becomes reliable.

17. FUTURE SCOPE

- Complete ISL alphabet recognition.
- Recognition of dynamic signs and common words.
- Sentence construction from sequential signs.
- Bidirectional communication by adding speech-to-text.
- Mobile or web deployment.
- Edge deployment on a suitable embedded/AI device.
- Improved multilingual or speech-output support.
- User-specific calibration while retaining a general model.

18. CONCLUSION

The Indian Sign Language Recognition System demonstrates a complete prototype pipeline from webcam input to landmark extraction, sequence formation, Transformer-based classification, on-screen prediction and text-to-speech output. The project moved beyond a simple static landmark approach by introducing temporal sequences and a MediaPipe Holistic representation intended to capture richer hand/body information.

The current implementation proves the main technical workflow for the A-E prototype. The next major requirement is not simply adding more code, but improving the dataset, class balance, prediction stability and evaluation methodology. With a larger and more diverse dataset and stronger inference gating, the same architecture can be extended toward a more reliable ISL communication system.

19. PROJECT STATUS AT THE TIME OF THIS README

Prototype stage: Functional real-time A-E recognition pipeline with Transformer inference and TTS integration.

Primary remaining work: Improve classification stability, especially the repeated-E behavior, and expand/strengthen the dataset before claiming high general-purpose accuracy.

Documentation principle: No unsupported accuracy percentage is stated here because the available project records do not establish a final, reliable test accuracy for the complete 260-feature real-time system.